

Семинар: Модули 0-2. От синхронности к асинхронности

Часть 0. Пролог: зачем это нужно

0.1. Синхронный мир и его предел

Когда сервер обрабатывает запрос синхронно, поток исполнения блокируется на время любой I/O-операции: чтения из базы данных, запроса к внешнему API, чтения файла с диска. Пока операция не завершится, поток ничего не делает.

Ключевые понятия:

- **I/O-bound задачи** — процессор ждет (сеть, диск, БД). В вебе их большинство.
- **CPU-bound задачи** — процессор считает (ML-инференс, математика).
- **C10K Problem** — проблема 10 000 одновременных соединений. 10 000 потоков потребляют 10-80 Гб памяти только на стеки, а ОС тратит больше времени на переключение контекста, чем на полезную работу.
- **GIL (Global Interpreter Lock)** — мьютекс в CPython, позволяющий только одному потоку выполнять Python-код. Многопоточность в Python эффективна только для I/O-bound задач.

0.2. Три подхода к масштабированию

Подход	Когда применять	Проблемы
Многопоточность (threading)	I/O-bound, синхронные библиотеки	GIL мешает CPU-bound; накладные расходы на потоки
Мультипроцессинг (multiprocessing)	CPU-bound (ML-инференс)	Дорогое создание процессов; сериализация через pickle
Асинхронность (asyncio)	I/O-bound, тысячи соединений	CPU-bound блокирует цикл; код сложнее

0.3. Почему FastAPI?

FastAPI стоит на трех столпах: **Starlette** (ASGI-фреймворк), **Pydantic V2** (валидация на Rust), **Python** (доступ к ML-экосистеме). Это позволяет обрабатывать тысячи I/O-операций конкурентно и при этом вызывать Python-библиотеки для ML.

Часть 1. Фундамент Python

1.1. Итераторы и генераторы

Протокол итератора состоит из двух методов: `__iter__` возвращает объект с `__next__`, а `__next__` выдает следующий элемент или `StopIteration`. Генератор — это функция с `yield`, которая приостанавливает выполнение, сохраняя локальное состояние.

```
In [1]: # Протокол итератора: разбор for-под капотом
data = [10, 20, 30]

# Эквивалент:
_iterator = iter(data)
while True:
    try:
        element = next(_iterator)
        print(element)
    except StopIteration:
        break
```

Out:

```
10
20
30
```

```
In [2]: # Свой итератор: квадраты чисел
class SquareIterator:
    def __init__(self, start: int, end: int):
        self.current = start
        self.end = end

    def __iter__(self):
        return self

    def __next__(self):
        if self.current > self.end:
            raise StopIteration
        value = self.current ** 2
```

```
        self.current += 1
        return value

sq = SquareIterator(1, 3)
print(list(sq)) # [1, 4, 9]
```

Out:

```
[1, 4, 9]
```

```
In [3]: # Генераторная функция: то же самое, но компактнее
def squares(start, end):
    current = start
    while current <= end:
        yield current ** 2
        current += 1

print(list(squares(1, 3))) # [1, 4, 9]

# Генераторное выражение: ленивый аналог list comprehension
squares_gen = (x**2 for x in range(1, 4))
print(list(squares_gen)) # [1, 4, 9]
```

Out:

```
[1, 4, 9]
[1, 4, 9]
```

```
In [4]: # yield from: делегирование другому итератору
def sub_generator():
    yield 1
    yield 2

def main_generator():
    yield "start"
    yield from sub_generator()
    yield "end"

print(list(main_generator())) # ['start', 1, 2, 'end']
```

Out:

```
['start', 1, 2, 'end']
```

```
In [5]: # Бесконечный генератор (ленивые вычисления)
class InfiniteCounter:
    def __init__(self, start=0):
        self.n = start
    def __iter__(self):
        return self
    def __next__(self):
        value = self.n
        self.n += 1
        return value

counter = InfiniteCounter()
result = []
for i in counter:
    if i > 5:
        break
    result.append(i)
print(result) # [0, 1, 2, 3, 4, 5]
```

```
Out:
[0, 1, 2, 3, 4, 5]
```

Ключевая связь: `await` в `asyncio` — это синтаксический сахар над `yield from` для корутин. Генераторы учат нас: функция может приостанавливаться и возобновляться, сохраняя состояние.

1.2. Менеджеры контекста

Протокол: `__enter__` — инициализация ресурса, `__exit__` — гарантированное освобождение. `__exit__` вызывается всегда, даже при исключении.

```
In [6]: import time
from typing import Optional

# Свой менеджер контекста: таймер
class Timer:
    def __enter__(self):
        self.start = time.perf_counter()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb) -> Optional[bool]:
        self.elapsed = time.perf_counter() - self.start
```

```
print(f"Прошло {self.elapsed:.4f} секунд")
return None # исключение не подавляем

with Timer():
    time.sleep(0.1)
```

Out:

```
Прошло 0.1006 секунд
```

```
In [7]: from contextlib import contextmanager

# Менеджер через генератор и декоратор
@contextmanager
def managed_resource(name):
    print(f"Ресурс {name}: инициализация")
    resource = {"name": name, "active": True}
    try:
        yield resource
    finally:
        resource["active"] = False
        print(f"Ресурс {name}: освобождение")

with managed_resource("database") as db:
    print(f"Работаем с {db}")
print(f"После выхода: active={db['active']}")
```

Out:

```
Ресурс database: инициализация
Работаем с {'name': 'database', 'active': True}
Ресурс database: освобождение
После выхода: active=False
```

```
In [8]: from contextlib import asynccontextmanager

# Асинхронный менеджер контекста
@asynccontextmanager
async def async_managed_resource(name):
    print(f"Асинх ресурс {name}: подключение")
    # Имитация асинхронного подключения
    conn = {"name": name, "connected": True}
    try:
        yield conn
```

```

finally:
    conn["connected"] = False
    print(f"Async ресурс {name}: отключение")

async def demo_async_context():
    async with async_managed_resource("postgres") as conn:
        print(f"Работаем с {conn}")
    print(f"После выхода: connected={conn['connected']}")

# Запуск
import asyncio
# asyncio.run(demo_async_context())
await demo_async_context()

```

Out:

```

Async ресурс postgres: подключение
Работаем с {'name': 'postgres', 'connected': True}
Async ресурс postgres: отключение
После выхода: connected=False

```

Ключевая связь: `async with` в FastAPI используется для управления соединениями с БД, сессиями, транзакциями. `lifespan` в FastAPI — это по сути `async with` над всем приложением.

1.3. Типизация в Python

Аннотации типов (PEP 484+) — подсказки для статического анализа. Pydantic превращает их в runtime-валидаторы.

```

In [9]: from typing import List, Dict, Optional, Union, Callable, TypeVar, Generic

# Базовые типы
def process(data: list[int]) -> dict[str, int]:
    return {"sum": sum(data)}

def find(user_id: int) -> Optional[str]:
    return "Alice" if user_id > 0 else None

def apply(func: Callable[[int], bool], items: list[int]) -> list[int]:
    return [x for x in items if func(x)]

# Generic и TypeVar
T = TypeVar('T')

```

```

class Stack(Generic[T]):
    def __init__(self) -> None:
        self._items: list[T] = []

    def push(self, item: T) -> None:
        self._items.append(item)

    def pop(self) -> T:
        return self._items.pop()

int_stack: Stack[int] = Stack()
int_stack.push(42)
# int_stack.push("oops") # муру выдаст ошибку
print(int_stack.pop()) # 42

```

Out:

42

In [10]:

```

from typing import Protocol

# Protocol: структурная типизация (duck typing для статического анализа)
class Drawable(Protocol):
    def draw(self) -> None: ...

class Circle:
    def draw(self) -> None:
        print("Drawing circle")

class Rectangle:
    def draw(self) -> None:
        print("Drawing rectangle")

def render(items: list[Drawable]) -> None:
    for item in items:
        item.draw()

render([Circle(), Rectangle()])

```

Out:

Drawing circle
Drawing rectangle

```
In [11]: from pydantic import BaseModel, Field

# Pydantic: типы как runtime-контракты
class User(BaseModel):
    id: int
    name: str = Field(min_length=1, max_length=100)
    email: str
    age: int = Field(ge=0, le=150)

# Автоматическое преобразование и валидация
user = User(id="42", name="Alice", email="alice@example.com", age=25)
print(user.id, type(user.id)) # 42 <class 'int'>

# Ошибка валидации:
# User(id="42", name="A", email="alice@example.com", age=-5)
```

```
Out:
42 <class 'int'>
```

Вариантность типов (кратко):

- **Ковариантность** (`Iterable[+T]`): только читаем. `list[Cat]` можно передать туда, где ожидается `Iterable[Animal]`.
- **Контравариантность** (`Callable[-T, None]`): только пишем/принимаем. Функция, принимающая `Animal`, подходит туда, где ожидается `Callable[[Cat], None]`.
- **Инвариантность** (`List[T]`): и читаем, и пишем. `list[Cat]` не является `list[Animal]`.

Часть 2. Асинхронное программирование

2.1. Конкурентность vs Параллелизм

- **Конкурентность** — задачи чередуются в одном потоке. Один повар готовит два блюда: пока вода закипает — нарезает овощи.
- **Параллелизм** — задачи выполняются одновременно на разных ядрах. Два повара на двух плитах.

`asyncio` — кооперативная многозадачность: задачи сами уступают управление в точках `await`.

2.2. Event Loop

Event loop — бесконечный цикл, который:

1. Берет готовые задачи из очереди.
2. Передает им управление до ближайшего `await`.
3. Если задач нет — засыпает, дожидаясь внешнего события (готовность сокета, таймаут).
4. Просыпается и повторяет.

```
In [12]: import asyncio

# Базовый пример: что делает asyncio.run()
async def main():
    print("Hello")
    await asyncio.sleep(1) # уступаем управление на 1 секунду
    print("World")

# asyncio.run(main())
await main()
```

```
Out:
Hello
World
```

```
In [13]: import asyncio
import time

# Почему time.sleep() убивает производительность
async def bad_handler():
    time.sleep(2) # БЛОКИРУЕТ весь поток! Все корутины встанут.
    return "done"

async def good_handler():
    await asyncio.sleep(2) # Уступает управление. Другие корутины работают.
    return "done"

async def demo():
    start = time.perf_counter()
    # Запускаем 3 "хороших" хендлера конкурентно
    await asyncio.gather(good_handler(), good_handler(), good_handler())
    print(f"Good: {time.perf_counter() - start:.2f}s") # ~2s, не 6s

    start = time.perf_counter()
    # Запускаем 3 "плохих" хендлера последовательно (блокируют друг друга)
```

```

    await asyncio.gather(bad_handler(), bad_handler(), bad_handler())
    print(f"Bad: {time.perf_counter() - start:.2f}s") # ~6s

# asyncio.run(demo())
await demo()

```

Out:

```

Good: 2.01s
Bad: 6.00s

```

2.3. Корутины, Future, Task

Объект	Что это
Coroutine	<code>async def</code> функция, еще не запущенная. Черновик.
Future	Контейнер для результата, который появится позже. Обещание.
Task	Корутина, обернутая в задачу и поставленная в event loop.

```

In [14]: import asyncio

async def say_hello():
    await asyncio.sleep(1)
    return "Hello"

async def demo_objects():
    # Корутина – объект, не выполняется
    coro = say_hello()
    print(type(coro)) # <class 'coroutine'>

    # Future – низкоуровневый примитив
    future = asyncio.Future()
    print(future.done()) # False
    future.set_result(42)
    print(future.result()) # 42

    # Task – корутина в event loop
    task = asyncio.create_task(say_hello())
    result = await task
    print(result) # Hello

```

```
# asyncio.run(demo_objects())
await demo_objects()
```

Out:

```
<class 'coroutine'>
False
42
Hello
```

```
C:\Users\igory\AppData\Local\Temp\ipykernel_24456\2537276848.py:24: RuntimeWarning: corou
tine 'say_hello' was never awaited
    await demo_objects()
RuntimeWarning: Enable tracemalloc to get the object allocation traceback
```

In [15]:

```
import asyncio

# Состояния задачи
async def lifecycle_demo():
    print("RUNNING")
    await asyncio.sleep(0) # уступаем, но сразу возвращаемся
    print("BACK")

async def demo_states():
    task = asyncio.create_task(lifecycle_demo())
    print(f"done: {task.done()}")      # False
    print(f"cancelled: {task.cancelled()}") # False

    await task
    print(f"done after await: {task.done()}") # True
    print(f"result: {task.result()}") # None

# asyncio.run(demo_states())
await demo_states()
```

Out:

```
done: False
cancelled: False
RUNNING
BACK
done after await: True
result: None
```

2.4. Параллельное выполнение корутин

```
In [16]: import asyncio

async def fetch(url: str, delay: float) -> str:
    await asyncio.sleep(delay)
    return f"Data from {url}"

# gather: конкурентный запуск, результаты в порядке аргументов
async def demo_gather():
    results = await asyncio.gather(
        fetch("A", 2),
        fetch("B", 1),
        fetch("C", 3),
    )
    print(results) # ['Data from A', 'Data from B', 'Data from C']

# asyncio.run(demo_gather())
await demo_gather()
```

```
Out:
['Data from A', 'Data from B', 'Data from C']
```

```
In [17]: import asyncio

# gather с обработкой исключений
async def failing_fetch(url: str) -> str:
    await asyncio.sleep(0.5)
    raise ValueError(f"Failed: {url}")

async def demo_gather_exceptions():
    results = await asyncio.gather(
        fetch("A", 1),
        failing_fetch("B"),
        fetch("C", 2),
        return_exceptions=True
    )
    print(results)
    # ['Data from A', ValueError('Failed: B'), 'Data from C']

# asyncio.run(demo_gather_exceptions())
await demo_gather_exceptions()
```

Out:

```
['Data from A', ValueError('Failed: B'), 'Data from C']
```

In [18]:

```
import asyncio

# wait: тонкое управление
async def demo_wait():
    tasks = [
        asyncio.create_task(fetch("A", 2)),
        asyncio.create_task(fetch("B", 1)),
    ]

    done, pending = await asyncio.wait(
        tasks,
        return_when=asyncio.FIRST_COMPLETED
    )
    print(f"Готово: {len(done)}, В ожидании: {len(pending)}")

    # Дожидаемся оставшихся
    if pending:
        await asyncio.wait(pending)

# asyncio.run(demo_wait())
await demo_wait()
```

Out:

```
Готово: 1, В ожидании: 1
```

In [19]:

```
import asyncio

# as_completed: обработка по мере готовности
async def demo_as_completed():
    coros = [fetch(f"url_{i}", i) for i in [3, 1, 2]]

    for coro in asyncio.as_completed(coros):
        result = await coro
        print(result)
        # url_1 (через 1с), url_2 (через 2с), url_3 (через 3с)

# asyncio.run(demo_as_completed())
await demo_as_completed()
```

Out:

```
Data from url_1
Data from url_2
Data from url_3
```

In [20]:

```
import asyncio

# create_task: фоновые задачи
async def background_worker():
    for i in range(3):
        print(f"Worker tick {i}")
        await asyncio.sleep(1)

async def demo_background():
    task = asyncio.create_task(background_worker())

    # Делаем что-то другое
    await asyncio.sleep(2.5)
    print("Main done")

    # Отменяем фоновую задачу
    task.cancel()
    try:
        await task
    except asyncio.CancelledError:
        print("Фоновая задача отменена")

# asyncio.run(demo_background())
await demo_background()
```

Out:

```
Worker tick 0
Worker tick 1
Worker tick 2
Main done
Фоновая задача отменена
```

In [21]:

```
import asyncio

# Таймауты
async def slow_operation():
    await asyncio.sleep(10)
    return "done"
```

```

async def demo_timeout():
    # Python 3.11+
    try:
        async with asyncio.timeout(2):
            await slow_operation()
    except asyncio.TimeoutError:
        print("Таймаут!")

    # До 3.11
    try:
        await asyncio.wait_for(slow_operation(), timeout=2.0)
    except asyncio.TimeoutError:
        print("Таймаут (wait_for)!")

# asyncio.run(demo_timeout())
await demo_timeout()

```

Out:

```

Таймаут!
Таймаут (wait_for)!

```

2.5. Синхронизация

```

In [22]: import asyncio

# Lock: взаимное исключение
lock = asyncio.Lock()
shared_counter = 0

async def increment():
    global shared_counter
    async with lock:
        current = shared_counter
        await asyncio.sleep(0) # имитация "тяжелой" операции
        shared_counter = current + 1

async def demo_lock():
    global shared_counter
    shared_counter = 0
    await asyncio.gather(*[increment() for _ in range(1000)])
    print(f"Counter: {shared_counter}") # 1000

```

```
# asyncio.run(demo_lock())
await demo_lock()
```

Out:

```
Counter: 1000
```

In [23]:

```
import asyncio

# Semaphore: ограничение параллелизма
semaphore = asyncio.Semaphore(3) # не более 3 одновременно

async def fetch_limited(url: str, delay: float) -> str:
    async with semaphore:
        print(f" Starting {url}")
        await asyncio.sleep(delay)
        print(f" Done {url}")
        return f"Data from {url}"

async def demo_semaphore():
    urls = [f"url_{i}" for i in range(10)]
    delays = [1.0] * 10
    results = await asyncio.gather(*[
        fetch_limited(u, d) for u, d in zip(urls, delays)
    ])
    print(f"Bcero: {len(results)}")

# asyncio.run(demo_semaphore())
await demo_semaphore()
```

Out:

```
Starting url_0
Starting url_1
Starting url_2
Done url_0
Done url_1
Done url_2
Starting url_3
Starting url_4
Starting url_5
Done url_3
Done url_4
Done url_5
Starting url_6
Starting url_7
```



```
Starting url_8
Done url_6
Done url_7
Done url_8
Starting url_9
Done url_9
Bcero: 10
```

```
In [24]: import asyncio

# Event: сигнализация между корутинами
event = asyncio.Event()

async def waiter(name: str):
    print(f"{name}: жду события...")
    await event.wait()
    print(f"{name}: событие произошло!")

async def setter():
    await asyncio.sleep(2)
    print("Setter: выставляю событие")
    event.set()

async def demo_event():
    await asyncio.gather(waiter("A"), waiter("B"), setter())

# asyncio.run(demo_event())
await demo_event()
```

```
Out:
A: жду события...
B: жду события...
Setter: выставляю событие
A: событие произошло!
B: событие произошло!
```

```
In [25]: import asyncio

condition = asyncio.Condition()
queue = []
done = False # Флаг завершения

async def consumer(name: str):
    async with condition:
```

```

while not queue and not done:
    await condition.wait()
if queue:
    item = queue.pop(0)
    print(f"{name}: ✅ обработано {item}")
else:
    print(f"{name}: ❌ завершаю работу (данных нет)")

async def producer():
    await asyncio.sleep(1)
    async with condition:
        queue.append("data")
        print("Producer: ➕ добавлено data")
        # Пробуждаем всех, но только один получит данные
        condition.notify_all()

async def demo_condition():
    # Ждем 2 секунды и устанавливаем флаг done
    await asyncio.gather(
        consumer("C1"),
        consumer("C2"),
        producer(),
        shutdown_after(2) # Таймаут для завершения
    )

async def shutdown_after(seconds: int):
    await asyncio.sleep(seconds)
    global done
    async with condition:
        done = True
        condition.notify_all()
    print("❌ Сигнал завершения отправлен")

# asyncio.run(demo_condition())
await demo_condition()

```

Out:

```

Producer: ➕ добавлено data
C1: ✅ обработано data
❌ Сигнал завершения отправлен
C2: ❌ завершаю работу (данных нет)

```

2.6. CPU-bound задачи в асинхронном коде

```
In [26]: import asyncio
import concurrent.futures

# ThreadPoolExecutor: для блокирующего I/O (освобождает event loop)
def sync_function(x: int) -> int:
    # Синхронная функция, которую нельзя переписать
    return x ** 2

async def demo_thread_pool():
    loop = asyncio.get_running_loop()
    with concurrent.futures.ThreadPoolExecutor() as pool:
        result = await loop.run_in_executor(pool, sync_function, 42)
        print(result) # 1764

# asyncio.run(demo_thread_pool())
await demo_thread_pool()
```

Out:

```
1764
```

```
In [27]: import asyncio
import concurrent.futures

# ProcessPoolExecutor: для CPU-bound задач (настоящий параллелизм)
def cpu_intensive(n: int) -> int:
    return sum(i * i for i in range(n))

async def demo_process_pool():
    loop = asyncio.get_running_loop()
    with concurrent.futures.ProcessPoolExecutor() as pool:
        result = await loop.run_in_executor(pool, cpu_intensive, 10_000_000)
        print(f"Result: {result}")

asyncio.run(demo_process_pool())
```

```
In [28]: import asyncio
from concurrent.futures import ProcessPoolExecutor

# Паттерн "основной цикл + пул воркеров" для ML
import numpy as np

_model = None
```

```

def init_worker():
    global _model
    # Имитация загрузки ML-модели
    _model = lambda x: x * 2 # заглушка

def predict(features: np.ndarray) -> np.ndarray:
    return _model(features)

_executor = ProcessPoolExecutor(max_workers=2, initializer=init_worker)

async def predict_endpoint(features: list[float]):
    loop = asyncio.get_running_loop()
    arr = np.array(features)
    result = await loop.run_in_executor(_executor, predict, arr)
    return result.tolist()

async def demo_ml_pattern():
    result = await predict_endpoint([1.0, 2.0, 3.0])
    print(f"Prediction: {result}")

asyncio.run(demo_ml_pattern())

```

2.7. Асинхронные генераторы и контекстные менеджеры

In [29]:

```

import asyncio

# Асинхронный генератор
async def async_range(n: int):
    for i in range(n):
        await asyncio.sleep(0.1) # имитация асинхронной работы
        yield i

async def demo_async_for():
    async for value in async_range(5):
        print(value, end=" ")
    print()

# asyncio.run(demo_async_for())
await demo_async_for()

```

Out:

0 1 2 3 4

```
In [30]: from contextlib import asynccontextmanager

# Асинхронный контекстный менеджер для транзакции
@asynccontextmanager
async def database_transaction():
    print("BEGIN transaction")
    try:
        yield {"conn": "postgres"}
    except Exception:
        print("ROLLBACK")
        raise
    else:
        print("COMMIT")

async def demo_async_with():
    async with database_transaction() as conn:
        print(f"Executing query on {conn['conn']}")
        # raise ValueError("oops") # раскомментируйте для проверки rollback

asyncio.run(demo_async_with())
```

```
In [31]: import asyncio

# Backpressure через asyncio.Queue
async def producer(queue: asyncio.Queue, n: int):
    for i in range(n):
        await queue.put(f"item_{i}")
        print(f"Produced: item_{i}")
        await asyncio.sleep(0.3)

async def consumer(queue: asyncio.Queue):
    while True:
        item = await queue.get()
        if item is None: # сигнал завершения
            break
        print(f"Consumed: {item}")
        await asyncio.sleep(0.5) # потребитель медленнее
        queue.task_done()

async def demo_backpressure():
    queue = asyncio.Queue(maxsize=3) # ограничение буфера
    prod_task = asyncio.create_task(producer(queue, 10))
    cons_task = asyncio.create_task(consumer(queue))
```

```
await prod_task
await queue.put(None) # сигнал завершения
await cons_task

# asyncio.run(demo_backpressure())
await demo_backpressure()
```

Out:

```
Produced: item_0
Consumed: item_0
Produced: item_1
Consumed: item_1
Produced: item_2
Produced: item_3
Consumed: item_2
Produced: item_4
Consumed: item_3
Produced: item_5
Produced: item_6
Consumed: item_4
Produced: item_7
Consumed: item_5
Produced: item_8
Consumed: item_6
Produced: item_9
Consumed: item_7
Consumed: item_8
Consumed: item_9
```

Задания для самостоятельной работы

Задание 1. Итератор с фильтрацией

Напишите класс `FilteredIterator`, который принимает `iterable` и функцию-предикат.

Итератор должен выдавать только те элементы, для которых предикат возвращает `True`.

Используйте протокол `__iter__` / `__next__`.

```
In [32]: class FilteredIterator():

        def __init__(self, iterable, predicate):
            self._iterator = iter(iterable)
            self._predicate = predicate
```

```

def __iter__(self):
    return self

def __next__(self):
    while True:
        element = next(self._iterator)
        if self._predicate(element):
            return element

numbers = [1, 2, 3, 4, 5, 6]
is_even = lambda x: x % 2 == 0

filtered = FilteredIterator(numbers, is_even)

for num in filtered:
    print(num, end=", ")

```

Out:

```
2, 4, 6,
```

In [33]:

```

class FilteredIterator:
    def __init__(self, iterable, predicate):
        self._iterator = iter(iterable)
        self._predicate = predicate
        self._buffer = None
        self._has_buffer = False

    def __iter__(self):
        return self

    def __next__(self):
        while True:
            item = next(self._iterator)
            if self._predicate(item):
                return item

# Проверка
data = [1, 2, 3, 4, 5, 6]
filt = FilteredIterator(data, lambda x: x % 2 == 0)
print(list(filt)) # [2, 4, 6]

```

Out:

```
[2, 4, 6]
```

Задание 2. Генератор Фибоначчи

Напишите генераторную функцию `fibonacci(n)`, которая выдает первые `n` чисел Фибоначчи. Затем напишите генераторное выражение, которое берет только четные числа из этого генератора.

In [34]:

```
def fibonacci(n: int):
    a, b = 0, 1
    cnt = 0
    while cnt < n:
        yield a
        a, b = b, a+b
        cnt += 1

filtered = (x for x in fibonacci(1000) if x % 100 == 0)
print(list(filtered))
```

Out:

```
[0, 9969216677189303386214405760200, 2222322446294204455297398934619099672066669390964997
64990979600, 4953967011875066473162524925231604047727791871346061001150551747313593851366
517214899257280600, 110433070572952242346432246767718285942590237357555606380008891875277
701705731473925618404421867819924194229142447517901959200, 246175702158232427216624815531
30368936971399966974615095762332110000556079121989797049887044464258340427952696035885222
45550271050495783935904220352228801000, 5487710883948000005141367394838371444380051930912
35927244949534270398112010643412349543875215253906155049490921874412182466791047314424730
22013980160407007017175697317900483275246652938800]
```

In [35]:

```
def fibonacci(n: int):
    a, b = 0, 1
    for _ in range(n):
        yield a
        a, b = b, a + b

# Первые 20 чисел Фибоначчи
fib_gen = fibonacci(20)
print(list(fib_gen))
# [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584,
4181]
```



```
# Только четные через генераторное выражение
even_fib = (x for x in fibonacci(20) if x % 2 == 0)
print(list(even_fib))
# [0, 2, 8, 34, 144, 610, 2584]
```

Задание 3. Менеджер контекста для таймера с логированием

Напишите менеджер контекста `TimedBlock`, который:

- При входе запоминает время.
- При выходе печатает, сколько прошло.
- Если в блоке произошло исключение — печатает время и пробрасывает исключение дальше.

```
In [36]: import time
from typing import Optional

class TimedBlock:
    def __init__(self, label: str = "Block"):
        self.label = label

    def __enter__(self):
        self.start = time.perf_counter()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb) -> Optional[bool]:
        elapsed = time.perf_counter() - self.start
        status = "OK" if exc_type is None else f"ERROR ({exc_type.__name__})"
        print(f"[{self.label}] {status} за {elapsed:.4f}s")
        return None # не подавляем исключение

# Проверка
with TimedBlock("Normal"):
    time.sleep(0.1)

with TimedBlock("Failing"):
    time.sleep(0.05)
    raise ValueError("oops") # исключение пробросится, но время напечатается
```

Out:

```
[Normal] OK за 0.1001s
[Failing] ERROR (ValueError) за 0.0508s
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[56], line 24
      22 with TimedBlock("Failing"):
      23     time.sleep(0.05)
--> 24     raise ValueError("oops") # исключение пробросится, но время напечатается

ValueError: oops
```

Задание 4. Pydantic-модель с валидацией

Создайте Pydantic-модель `Product` с полями:

- `name` (str, min_length=1)
- `price` (float, ge=0)
- `quantity` (int, ge=0)
- `tags` (list[str], опционально)

Продемонстрируйте автоматическое преобразование типов и валидацию.

```
In [37]: from pydantic import BaseModel, Field, ValidationError
         from typing import Optional

         class Product(BaseModel):

             name: str = Field(min_length=1)
             price: float = Field(ge=0)
             quantity: int = Field(ge=0)
             tags: Optional[list[str]] = None
```

```
In [38]: from pydantic import BaseModel, Field, ValidationError
         from typing import Optional

         class Product(BaseModel):
             name: str = Field(min_length=1)
             price: float = Field(ge=0)
```

```

quantity: int = Field(ge=0)
tags: Optional[list[str]] = None

# Корректные данные с автопреобразованием
p1 = Product(name="Laptop", price="999.99", quantity="5", tags=["electronics", "computers"])
print(p1)
# name='Laptop' price=999.99 quantity=5 tags=['electronics', 'computers']

# Некорректные данные
try:
    p2 = Product(name="", price=-10, quantity=-1)
except ValidationError as e:
    print(e)

```

Out:

```

name='Laptop' price=999.99 quantity=5 tags=['electronics', 'computers']
3 validation errors for Product
name
  String should have at least 1 character [type=string_too_short, input_value='', input_type=str]
  For further information visit https://errors.pydantic.dev/2.13/v/string_too_short
price
  Input should be greater than or equal to 0 [type=greater_than_equal, input_value=-10, input_type=int]
  For further information visit https://errors.pydantic.dev/2.13/v/greater_than_equal
quantity
  Input should be greater than or equal to 0 [type=greater_than_equal, input_value=-1, input_type=int]
  For further information visit https://errors.pydantic.dev/2.13/v/greater_than_equal

```

Задание 5. Конкурентный сбор данных

Напишите функцию `fetch_all(urls)`, которая конкурентно "загружает" данные из списка URL. Используйте `asyncio.gather`. Каждый URL "загружается" через `asyncio.sleep(random.uniform(0.5, 2.0))`. Верните результаты в том же порядке, в каком переданы URL.

```

In [39]: import asyncio
import random

async def fetch(url: str) -> str:
    delay = random.uniform(0.5, 2.0)

```

```

    await asyncio.sleep(delay)
    return f"Data from {url} (delay={delay:.2f}s)"

async def fetch_all(urls: list[str]) -> list[str]:
    tasks = [fetch(url) for url in urls]
    result = await asyncio.gather(*tasks)
    return result

async def main():
    urls = ["A", "B", "C"]
    res = await fetch_all(urls)
    print(*res)

await main()

```

Out:

```
Data from A (delay=1.31s) Data from B (delay=1.35s) Data from C (delay=1.74s)
```

In [40]:

```

import asyncio
import random

async def fetch(url: str) -> str:
    delay = random.uniform(0.5, 2.0)
    await asyncio.sleep(delay)
    return f"Data from {url} (delay={delay:.2f}s)"

async def fetch_all(urls: list[str]) -> list[str]:
    tasks = [fetch(url) for url in urls]
    return await asyncio.gather(*tasks)

async def main():
    urls = ["https://api.a.com", "https://api.b.com", "https://api.c.com"]
    results = await fetch_all(urls)
    for r in results:
        print(r)

asyncio.run(main())

```

Задание 6. Ограничение параллелизма через Semaphore

Модифицируйте `fetch_all` из задания 5: добавьте `asyncio.Semaphore(2)`, чтобы одновременно выполнялось не более 2 запросов. Добавьте `print` при старте и завершении каждого запроса, чтобы было видно ограничение.

```

In [41]: import asyncio
import random

sema = asyncio.Semaphore(2)

async def fetch_limited(url: str) -> str:
    async with sema:

        print(f"Start fetch({url})")
        delay = random.uniform(0.5, 2.0)
        await asyncio.sleep(delay)
        print(f"End fetch({url}) with delay={delay:.2f}")

        return f"Data from {url}"

async def fetch_all_limited(urls: list[str]) -> list[str]:

    tasks = [fetch_limited(url) for url in urls]
    result = await asyncio.gather(*tasks)
    return result

async def main():
    urls = [letter for letter in "ABCDEFGH"]
    res = await fetch_all_limited(urls)
    print(res)

await main()

```

```

Out:
Start fetch(A)
Start fetch(B)
End fetch(B) with delay=0.78
Start fetch(C)
End fetch(C) with delay=0.58
Start fetch(D)
End fetch(A) with delay=1.89
Start fetch(E)
End fetch(D) with delay=0.59
Start fetch(F)
End fetch(E) with delay=1.51
Start fetch(G)
End fetch(F) with delay=1.97
End fetch(G) with delay=1.73
['Data from A', 'Data from B', 'Data from C', 'Data from D', 'Data from E', 'Data from F', 'Data from G']

```

```
In [42]: import asyncio
import random

semaphore = asyncio.Semaphore(2)

async def fetch_limited(url: str) -> str:
    async with semaphore:
        print(f"  START {url}")
        delay = random.uniform(0.5, 2.0)
        await asyncio.sleep(delay)
        print(f"  DONE  {url}")
        return f"Data from {url}"

async def fetch_all_limited(urls: list[str]) -> list[str]:
    tasks = [fetch_limited(url) for url in urls]
    return await asyncio.gather(*tasks)

async def main():
    urls = [f"https://api.{c}.com" for c in "abcde"]
    results = await fetch_all_limited(urls)
    print(f"\nВсего загружено: {len(results)}")

asyncio.run(main())
```

Задание 7. Отмена задачи с корректной очисткой

Напишите корутину `worker()`, которая в цикле каждую секунду печатает "tick". Запустите её как фоновую задачу. Через 3.5 секунды отмените задачу. Убедитесь, что корутина корректно обрабатывает отмену (делает cleanup) и пробрасывает `CancelledError`.

```
In [43]: import asyncio

async def worker():
    try:
        i = 0
        while True:
            print(f"Tick {i}")
            await asyncio.sleep(1)
            i += 1
    except asyncio.CancelledError:
        print("Cleanup: закрываем ресурсы...")
        raise # пробрасываем

async def main():
```

```

task = asyncio.create_task(worker())
await asyncio.sleep(3.5)
print("Main: отменяем worker")
task.cancel()
try:
    await task
except asyncio.CancelledError:
    print("Main: worker успешно отменен")

await main()

```

Out:

```

Tick 0
Tick 1
Tick 2
Tick 3
Main: отменяем worker
Cleanup: закрываем ресурсы...
Main: worker успешно отменен

```

In [44]:

```

import asyncio

async def worker():
    try:
        i = 0
        while True:
            print(f"tick {i}")
            await asyncio.sleep(1)
            i += 1
    except asyncio.CancelledError:
        print("Cleanup: закрываем ресурсы...")
        raise # обязательно пробрасываем!

async def main():
    task = asyncio.create_task(worker())
    await asyncio.sleep(3.5)
    print("Main: отменяем worker")
    task.cancel()
    try:
        await task
    except asyncio.CancelledError:
        print("Main: worker успешно отменен")

asyncio.run(main())

```

Задание 8. Таймаут с обработкой

Напишите функцию `fetch_with_timeout(url, timeout)`, которая "загружает" URL через `asyncio.sleep(5)` (имитация медленного сервера). Если загрузка занимает больше `timeout` секунд — должна выброситься `TimeoutError`, а не ждать вечно. Используйте `asyncio.timeout` (3.11+) или `asyncio.wait_for`.

```
In [45]: import asyncio

async def slow_fetch(url: str) -> str:
    await asyncio.sleep(5) # медленный сервер
    return f"Data from {url}"

async def fetch_with_timeout(url: str, timeout: float) -> str:
    async with asyncio.timeout(timeout):
        return await slow_fetch(url)

async def main():
    try:
        result = await fetch_with_timeout("https://slow.api.com", timeout=2.0)
        print(result)
    except asyncio.TimeoutError:
        print("Таймаут! Сервер не ответил за 2 секунды")

asyncio.run(main())
```

Задание 9. Race condition и его устранение

Напишите две версии `increment_counter(n)`: одну без `asyncio.Lock` и одну с `asyncio.Lock`. Запустите 1000 корутин, каждая из которых инкрементирует глобальный счетчик. Сравните результаты.

```
In [46]: import asyncio

# Версия БЕЗ lock (race condition)
counter_no_lock = 0

async def increment_no_lock():
    global counter_no_lock
    current = counter_no_lock
    await asyncio.sleep(0) # точка уступки – здесь происходит интерливинг
    counter_no_lock = current + 1
```



```

# Версия C lock
counter_with_lock = 0
lock = asyncio.Lock()

async def increment_with_lock():
    global counter_with_lock
    async with lock:
        current = counter_with_lock
        await asyncio.sleep(0)
        counter_with_lock = current + 1

async def main():
    # Без lock
    global counter_no_lock, counter_with_lock
    counter_no_lock = 0
    await asyncio.gather(*[increment_no_lock() for _ in range(1000)])
    print(f"Без lock: {counter_no_lock} (ожидалось 1000)")

    # C lock
    counter_with_lock = 0
    await asyncio.gather(*[increment_with_lock() for _ in range(1000)])
    print(f"C lock: {counter_with_lock} (ожидалось 1000)")

await main()

```

Out:

```

Без lock: 1 (ожидалось 1000)
C lock: 1000 (ожидалось 1000)

```

```

C:\Users\igory\AppData\Local\Python\pythoncore-3.14-64\Lib\asyncio\base_events.py:817: Ru
ntimeWarning: coroutine 'demo_process_pool' was never awaited
    def call_soon(self, callback, *args, context=None):
RuntimeWarning: Enable tracemalloc to get the object allocation traceback

```

Задание 10. CPU-bound задача в ProcessPoolExecutor

Напишите функцию `is_prime(n)`, которая проверяет, является ли число простым (наивный алгоритм). Запустите проверку для списка чисел `[10**6 + i for i in range(20)]` двумя способами:

1. Синхронно в цикле (замерьте время).
2. Через `ProcessPoolExecutor` с конкурентным выполнением (замерьте время).

```
In [47]: import asyncio
import concurrent.futures
import time

def is_prime(n: int) -> bool:
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    for i in range(3, int(n**0.5) + 1, 2):
        if n % i == 0:
            return False
    return True

def sync_check(numbers: list[int]) -> list[bool]:
    return [is_prime(n) for n in numbers]

async def async_check(numbers: list[int]) -> list[bool]:
    loop = asyncio.get_running_loop()
    with concurrent.futures.ProcessPoolExecutor() as pool:
        futures = [loop.run_in_executor(pool, is_prime, n) for n in numbers]
        return await asyncio.gather(*futures)

async def main():
    numbers = [10**6 + i for i in range(20)]

    # Синхронно
    start = time.perf_counter()
    sync_results = sync_check(numbers)
    sync_time = time.perf_counter() - start
    print(f"Синхронно: {sync_time:.3f}s")

    # Через ProcessPool
    start = time.perf_counter()
    async_results = await async_check(numbers)
    async_time = time.perf_counter() - start
    print(f"ProcessPool: {async_time:.3f}s")

    print(f"Результаты совпадают: {sync_results == async_results}")

asyncio.run(main())
```

Задание 11. Асинхронный генератор для постраничной загрузки

Напишите асинхронный генератор `fetch_pages(base_url, max_pages)`, который "загружает" страницы данных. Каждая страница загружается через `asyncio.sleep(0.2)` и возвращает список из 5 элементов. Генератор останавливается, когда страница пуста или достигнут `max_pages`.

```
In [48]: import asyncio

async def fetch_pages(base_url: str, max_pages: int = 10):
    for page in range(1, max_pages + 1):
        await asyncio.sleep(0.2) # имитация сетевого запроса
        # Имитация: страницы 1-3 содержат данные, страница 4+ — пустые
        if page > 3:
            break
        data = [f"{base_url}/page{page}/item{i}" for i in range(5)]
        yield data

async def main():
    async for page_data in fetch_pages("https://api.example.com", max_pages=5):
        print(f"Page: {page_data}")

asyncio.run(main())
```

Задание 12. Асинхронный контекстный менеджер с rollback

Напишите `asynccontextmanager` `transaction`, который:

- При входе печатает "BEGIN".
- При нормальном выходе печатает "COMMIT".
- При исключении печатает "ROLLBACK" и пробрасывает исключение.

```
In [49]: from contextlib import asynccontextmanager

@asynccontextmanager
async def transaction(tx_id: str):
    print(f"[{tx_id}] BEGIN")
    try:
        yield tx_id
    except Exception as e:
        print(f"[{tx_id}] ROLLBACK: {e}")
        raise
```

```

else:
    print(f"[{tx_id}] COMMIT")

async def main():
    # Успешная транзакция
    async with transaction("T1") as tx:
        print(f" Working in {tx}")

    # Неуспешная транзакция
    try:
        async with transaction("T2") as tx:
            print(f" Working in {tx}")
            raise ValueError("Something went wrong")
    except ValueError:
        print(" Exception caught outside")

asyncio.run(main())

```

Итоговая сводка

Концепция	Модуль	Ключевой API / Паттерн
Синхронность vs Асинхронность	0	I/O-bound vs CPU-bound, C10K
GIL	0	threading бесполезен для CPU-bound
Протокол итератора	1	<code>__iter__</code> , <code>__next__</code> , <code>StopIteration</code>
Генераторы	1	<code>yield</code> , <code>yield from</code> , генераторные выражения
Менеджеры контекста	1	<code>__enter__</code> / <code>__exit__</code> , <code>@contextmanager</code>
Типизация	1	<code>Generic</code> , <code>TypeVar</code> , <code>Protocol</code> , <code>Pydantic</code>
Event Loop	2	<code>asyncio.run()</code> , <code>get_running_loop()</code>
Корутины	2	<code>async def</code> , <code>await</code>
Task / Future	2	<code>create_task()</code> , <code>asyncio.Future</code>
Конкурентный запуск	2	<code>gather()</code> , <code>wait()</code> , <code>as_completed()</code>

Концепция	Модуль	Ключевой API / Паттерн
Синхронизация	2	<code>Lock</code> , <code>Semaphore</code> , <code>Event</code> , <code>Condition</code>
Отмена и таймауты	2	<code>task.cancel()</code> , <code>asyncio.timeout()</code>
CPU-bound в async	2	<code>run_in_executor()</code> + <code>ProcessPoolExecutor</code>
Асинхронные генераторы	2	<code>async for</code> , <code>__aiter__</code> , <code>__anext__</code>
Асинхронные контексты	2	<code>async with</code> , <code>@asynccontextmanager</code>

Чек-лист для самопроверки

- ☐ Я могу объяснить, почему `time.sleep()` в `async def` блокирует весь сервер.
- ☐ Я понимаю разницу между `Coroutine` , `Task` и `Future` .
- ☐ Я умею использовать `asyncio.gather` для конкурентного выполнения.
- ☐ Я знаю, когда нужен `Lock` , а когда — `Semaphore` .
- ☐ Я умею отменять задачи и обрабатывать `CancelledError` корректно.
- ☐ Я понимаю, куда девать CPU-bound код в асинхронном приложении.
- ☐ Я могу написать `async for` и `async with` с нуля.